

2) question 2 code:

```
#include "EListNode.h"
#include "EList.h"

// Linked list
EList :: EList() : pHead(nullptr), numNodes(0)
{
}

// insert node with value val after a node
// if node is NULL, insert to the front
void EList :: Insert(int val, EListNode *pPre)
{
    EListNode *pNew = new EListNode(val);

    if (pPre == nullptr) {
        // Insert at front
        pNew->SetNext(pHead);
        pHead = pNew;
    } else {
        // Insert after pPre
        pNew->SetNext(pPre->GetNext());
        pPre->SetNext(pNew);
    }

    numNodes++;
}

// delete a node
void EList :: Delete(EListNode *pNode)
{
    if (pNode == nullptr || pHead == nullptr)
        return;

    if (pNode == pHead) {
        pHead = pHead->GetNext();
        delete pNode;
        numNodes--;
        return;
    }

    EListNode *pPrev = pHead;
    while (pPrev != nullptr && pPrev->GetNext() != pNode) {
        pPrev = pPrev->GetNext();
    }

    if (pPrev != nullptr && pPrev->GetNext() == pNode) {
        pPrev->SetNext(pNode->GetNext());
        delete pNode;
        numNodes--;
    }
}

// get a node with value; if multiple nodes with the same value, return the first from head
EListNode * EList::GetNode(int val)
{
    EListNode *p = pHead;
    while (p != nullptr) {
        if (p->GetValue() == val)
            return p;
        p = p->GetNext();
    }
    return nullptr;
}
```

```

// get the number of nodes in the list
int EList:: GetSize() const
{
    return numNodes;
}

#ifndef _EC_LIST_NODE_H
#define _EC_LIST_NODE_H

#include <iostream>

// Class for linked list node
class EListNode
{
public:
    EListNode() : value(0), pNext(nullptr) {}
    EListNode(int v) : value(v), pNext(nullptr) {}

    EListNode *GetNext() const { return pNext; }
    void SetNext(EListNode *pn) { pNext = pn; }

    int GetValue() const { return value; }
    void SetValue(int v) { value = v; }

private:
    int value;
    EListNode *pNext;
};

#endif

```

output:

```

karin@Karinas_Laptop CLANG64 /c/Users/karin/Music/CSE-3150/lab5
$ clang++ EListTest.cpp EList.cpp

karin@Karinas_Laptop CLANG64 /c/Users/karin/Music/CSE-3150/lab5
$ ./a.exe
Number of nodes: 3
Number of nodes: 2

```

5) question 5 code:

```
// Note: you need to add compiler flag -fno-elide-constructors to
// disable copy elision (return value optimization) in order to see
// move constructor work as what you expect
#include "ECMatrix.h"
#include <iostream>

// default constructor
ECMatrix::ECMatrix()
{
}

// create a matrix of nr rows and nc columns
ECMatrix::ECMatrix(int nr, int nc)
{
    // your code
    std::cout << "Parameterized constructor called\n";
    listElements = std::vector<std::vector<double>>(nr, std::vector<double>(nc, 0.0));
}

// Copy constructor
ECMatrix::ECMatrix(const ECMatrix &rhs)
{
    // your code
    std::cout << "Copy constructor called\n";
    listElements = rhs.listElements;
}

//
ECMatrix ECMatrix::operator=(const ECMatrix &rhs)
{
    // your code
    std::cout << "Copy assignment operator called\n";
    if (this != &rhs)
    {
        listElements = rhs.listElements;
    }
    return *this;
}

// Move constructor
ECMatrix::ECMatrix(ECMatrix &&rhs)
{
    // your code
    std::cout << "Move constructor called\n";
    listElements = std::move(rhs.listElements);
    rhs.listElements.clear();
}

// Get num of rows/columns
int ECMatrix::GetNumRows() const
{
    // your code
    return listElements.size();
}

int ECMatrix::GetNumCols() const
{
    // your code
    if (listElements.empty())
        return 0;
    return listElements[0].size();
}

// Get/set an element in the matrix
double ECMatrix::GetVal(int r, int c) const
{
}
```

```

    // your code
    return listElements[r][c];
}
void EMatrix::SetVal(int r, int c, double val)
{
    // your code
    listElements[r][c] = val;
}

// Scale by a factor
EMatrix EMatrix:: Scale(double factor) const
{
    // your code
    std::cout << "Scale() called\n";
    EMatrix result(GetNumRows(), GetNumCols());
    for (int i = 0; i < GetNumRows(); ++i)
    {
        for (int j = 0; j < GetNumCols(); ++j)
        {
            result.SetVal(i, j, GetVal(i, j) * factor);
        }
    }
    return result;
}

```

output:

```

karin@Karin's Laptop CLANG64 /c/Users/karin/Music/CSE-3150/lab5:
$ clang++ EMatrix.cpp EMatrixTest.cpp

karin@Karin's Laptop CLANG64 /c/Users/karin/Music/CSE-3150/lab5:
$ ./a.exe
Parameterized constructor called
scale() called
Parameterized constructor called
scale() called
Parameterized constructor called
valsum: 1
Elapsed time (microseconds): 5736719

```